```
const int speakerPin = 5;
const int sensorPin = 2;
```

These first two lines define which pins are connected to which device. In this case, we have connected the PIR motion detector to pin 2 and the speaker is connected to pin 5.

```
const int highChirp = 20;
const int lowChirp = 14;
```

Crickets chirp fairly fast. These second two lines set up the length of time (measured in milliseconds) that the chirp is on, using the variable `highChirp`, and off, using the variable `lowChirp`. With our numbers, the chirp will be on for a little bit longer than it is off, giving the chirp a more natural irregularity.

```
const int chirpCycle = 70;
const int chirpPause = 8;
const int numChirps = 10;
const int midChirp = 150;
const int skittish = 5000;
```

These five variables are what give our little cricket its character. The variable `chirpCycle` is used in a block of code that will give the chirp a higher frequency of sound than if we simply turned the pin on. `chirpCycle` refers to the duty cycle that the pin is on for every 1 millisecond, measured in a percentage or 70% , in our example. Increasing or decreasing this percentage will affect the pitch of the sound generated. `chirpPause` is used after the main chirp sequence to give the cricket a 1 in 8 chance of pausing for a longer time than normal. In our chirp sequence, `numChirps` determines the number of chirps in a cycle. The value 10 works for a good cricket sound but increasing or decreasing this will create a cricket that is more or less active. `midChirp` represents a brief pause measured in milliseconds in the middle of the chirp cycle. The final variable, `skittish`, is how long the cricket will stay quiet after movement has been detected. Next, we move on to our `setup()` function:

```
pinMode(speakerPin, OUTPUT);
pinMode(sensorPin, INPUT);
```

After all of the variables have been defined, our `setup()` function will set each of the I/O pins according to whether we intend to use them as inputs or outputs. More on this later, so let's move on to our `loop()` function.

```
while (digitalRead(sensorPin) == LOW) {
  for (int i=1; i<=numChirps; i++) {
```

The first line of our `loop()` function reads the state of the sensor pin and as long as it remains `LOW`, meaning that movement has not been detected, the second line will begin the chirp cycle for as many chirps that have been specified.

```
for (int j=1; j<=highChirp; j++) {
  digitalWrite(speakerPin, HIGH);
  delayMicroseconds(chirpCycle * 10);
  digitalWrite(speakerPin, LOW);
  delayMicroseconds(1000-(chirpCycle * 10));
}
```

This entire block of code is a replacement for turning on the speaker pin once. So why use five lines of code instead of one? The reason is to create a higher pitch sound by turning on and off the pin at a higher rate of speed, or frequency. This `for` loop will cycle through once each millisecond for a number